

Generative & Agentic AI

20 Industry-Flavored Company Micro-Internship Modules

Teach-AI Research & Engineering Team

August 6, 2026

Abstract

This document outlines 20 hands-on micro-internship modules designed to bridge the gap between academic GenAI concepts and enterprise production engineering. Each module reframes a core Generative AI, RAG, LangChain, LangGraph, or Model Context Protocol (MCP) pattern into a real-world company engineering assignment complete with industry updates, technical deliverables, evaluation rubrics, and conceptual review tests.

Contents

1	Industry & Internship Index	2
2	Detailed Micro-Internship Modules	3

1 Industry & Internship Index

#	Target Company & Sector	Internship Role Title	Core Technology Scope
1	GlobalTech Solutions (IT Services)	Localization AI Intern	LCEL Chains, Translation Pipelines
2	ShopVerse (Retail & E-Commerce)	Catalog Automation Engineer	Pydantic Structured Outputs
3	ShieldCover Insurance (InsurTech)	Claims Workflow Intern	Parallel & Conditional Chains
4	MedCare Systems (Healthcare)	Clinical RAG Engineer	ChromaDB Vector Search & Embeddings
5	LexisClause Legal (Legal Tech)	Knowledge Systems Intern	Metadata-Filtered RAG Retrieval
6	Vertex National Bank (Banking)	Conversational AI Intern	LangGraph SQLite Checkpoint Memory
7	OmniRoute Logistics (Supply Chain)	Agentic Workflows Intern	ReAct Agent Pattern from Scratch
8	Apex Strategy (Consulting)	AI Advisory Intern	Reflexion Self-Critique Agent Loop
9	Apex Dynamics (Smart Manufacturing)	Multi-Agent Systems Intern	Hierarchical Supervisor Agent
10	QuantCapital (Financial Services)	Governance & Risk AI Intern	Human-in-the-Loop Approval Interruption
11	BioMedix Devices (Medical Equipment)	RAG Evaluation Intern	Text Chunking Comparison Strategy
12	TelcoNet Systems (Telecommunications)	MCP Protocol Engineer Intern	Python MCP Server (File Management)
13	DriveTech Motors (Automotive)	Full-Stack AI Engineer Intern	FastAPI + Gemini + MCP Diagnostics
14	WanderLust Global (Tourism)	Personalization AI Intern	Mem0 Hybrid Long-Term Memory
15	AeroSpace Dynamics (Aerospace)	Flight Ops AI Intern	DeepSeek/Gemini Quantitative Math JSON
16	FlavorCraft Foods (Food & Beverage)	Prompt Engineer Intern	Dual-LLM Comparison & Templating
17	QuickBite QSR (Restaurants)	Cloud AI Intern	Firebase Firestore Chat Persistence
18	SkyWay Travel (Tourism Tech)	TypeScript MCP Developer	Node.js/TypeScript Weather MCP Server
19	PriceWatch Retail (Market Intel)	Web Intelligence Agent Intern	Serper API Web Scraping Agent
20	OmniInsurance (Enterprise Insurance)	Enterprise Agent Architect	Nested Subgraph Architecture

Table 1: Overview of 20 Company Micro-Internships and Technical Focus Areas

2 Detailed Micro-Internship Modules

Module 1: LCEL Chain Building for Automated Localization

Company: GlobalTech Solutions (IT Services & Consulting)

Role: Localization AI Engineering Intern

Industry Context & Update: Global IT service leaders need automated localization pipelines to parse technical documentation, generate structured bullet points, and convert them across global offshore languages without manual overhead.

Problem Statement

Build a multi-stage LangChain Expression Language (LCEL) chain that takes a complex technical topic (e.g., "Kubernetes Autoscaling"), extracts 3 factual summary points in English, and translates those points into target regional languages (e.g., French or Japanese) in a unified pipeable workflow.

Key Learning Outcomes

- Master LCEL composition using the pipe operator (`|`).
- Utilize `ChatPromptTemplate` for dynamic multi-stage prompts.
- Implement `StrOutputParser` for seamless data transformations.

Tasks & Expected Deliverables

1. Construct a 3-stage chain (`Topic` → `Fact Generation` → `Translation`).
2. Execute end-to-end runs across 3 distinct technical topics.
3. Submit a working Jupyter Notebook (`chain.ipynb`) and a brief architecture explanation.

Knowledge Evaluation

1. What does LCEL stand for and why is it preferred over legacy chain classes?
2. Explain the difference between `RunnableSequence` and explicit `|` syntax.
3. What function does `StrOutputParser` perform in an LCEL pipeline?

Module 2: Structured Product Catalog Ingestion

Company: ShopVerse Marketplace (Retail & E-Commerce)

Role: Catalog AI Automation Intern

Industry Context & Update: E-Commerce platforms process millions of unstructured product descriptions submitted by third-party sellers. Production catalog databases mandate strict, type-safe JSON formats.

Problem Statement

Design a LangChain pipeline using Pydantic models to convert unstructured paragraph descriptions into validated JSON schema entries containing attributes such as `title`, `category`, `price_tier`, and key summary features.

Key Learning Outcomes

- Enforce structured LLM output using Pydantic `BaseModel`.
- Leverage `.with_structured_output()` for *schemabinding*.
- Handle validation errors and missing attributes gracefully.

Tasks & Expected Deliverables

1. Define a Pydantic schema with appropriate field validations.
2. Process 3 raw seller product descriptions into JSON catalog objects.
3. Submit notebook (`structured_catalog.ipynb`) and generated JSON outputs.

Knowledge Evaluation

1. Why is Pydantic structured output superior to asking LLMs to "return valid JSON" in prompt text?
2. How does LangChain handle schema mismatch or parsing exceptions during structured output calls?

Module 3: Parallel Claims Assessment & Conditional Routing

Company: ShieldCover Insurance (InsurTech)

Role: Claims Workflow Automation Intern

Industry Context & Update: InsurTech leaders accelerate claims processing by executing damage evaluation and fraud detection concurrently, followed by automated triage routing.

Problem Statement

Build a dual-chain system using `RunnableParallel` for concurrent damage assessment and fraud-risk calculation, combined with `RunnableBranch` to route claims conditionally to either "Auto-Approve" or "Manual Audit".

Key Learning Outcomes

- Execute concurrent LLM calls with `RunnableParallel`.
- Implement conditional logic routing using `RunnableBranch`.

Tasks & Expected Deliverables

1. Construct a parallel analysis execution node.
2. Implement conditional routing logic based on risk score thresholds.
3. Submit notebook (`parallel_claims.ipynb`) demonstrating both auto-approval and manual-review branches.

Knowledge Evaluation

1. When should developers select parallel execution over sequential chaining?
2. How does `RunnableBranch` evaluate condition predicates to determine route selection?

Module 4: Clinical Protocol RAG Pipeline with ChromaDB

Company: MedCare Health Systems (Healthcare & Clinical QA)

Role: Clinical RAG Systems Engineer Intern

Industry Context & Update: Hospital networks require context-grounded AI search over proprietary clinical guidelines to eliminate hallucination in medical queries and enable auditability with citations.

Problem Statement

Develop a Retrieval-Augmented Generation (RAG) system utilizing ChromaDB vector store to answer clinical questions grounded exclusively in internal guideline PDFs, highlighting exact source citations.

Key Learning Outcomes

- Implement text document chunking and vector embeddings.
- Build vector similarity retrieval workflows using ChromaDB.
- Construct context-injected prompt chains to guarantee factual grounding.

Tasks & Expected Deliverables

1. Ingest and chunk a sample clinical protocol document.
2. Query ChromaDB for top-k matching passages and return grounded answers with citations.
3. Submit notebook (`clinical_rag.ipynb`) and 3 sample Q&A outputs with citations.

Knowledge Evaluation

1. Why is RAG preferred over model fine-tuning for dynamic clinical protocol search?
2. What role does cosine similarity distance play during vector store retrieval?

Module 5: Metadata-Filtered Legal Precedent Retrieval

Company: LexisClause Legal Partners (Legal Tech)

Role: Legal Knowledge Systems Intern

Industry Context & Update: Law firms manage vast legal databases spanning jurisdictions, courts, and filing years. Pure semantic search often returns irrelevant precedents from incorrect legal jurisdictions.

Problem Statement

Extend a RAG pipeline by tagging document chunks with metadata (**jurisdiction**, **court**, **year**) and performing metadata-filtered vector retrieval to restrict searches strictly to relevant legal domains.

Key Learning Outcomes

- Embed documents with structured metadata dictionaries.
- Perform exact-match metadata filtering in vector searches.

Tasks & Expected Deliverables

1. Embed case law documents tagged with court and year metadata.
2. Run comparative queries showing search results with vs. without metadata filtering.
3. Submit notebook (`legal_filtered_rag.ipynb`) showing filtering accuracy.

Knowledge Evaluation

1. Why does pure semantic search fail when searching multi-jurisdiction legal archives?
2. Describe how vector databases execute metadata filter predicate evaluation.

Module 6: Banking Chatbot with Persistent Memory (LangGraph)

Company: Vertex National Bank (Banking & FinTech)

Role: Conversational AI Systems Intern

Industry Context & Update: Financial customer support bots must recall multi-turn loan details across user sessions. LangGraph state graphs with durable SQLite checkpointing ensure state persistence.

Problem Statement

Implement a stateful banking support chatbot using LangGraph `StateGraph` and `SqliteSaver` checkpointing that persists loan application memory across script restarts and user disconnects.

Key Learning Outcomes

- Understand stateful graph architectures in LangGraph.
- Implement durable session memory using SQLite checkpointers.

Tasks & Expected Deliverables

1. Design state schema tracking message history and user account state.
2. Demonstrate thread state persistence by closing and restarting the application session.
3. Submit notebook (`persistent_banking_bot.ipynb`) and memory recall trace.

Knowledge Evaluation

1. What is a checkpoint in LangGraph and how does it differ from in-memory arrays?
2. How are thread IDs utilized to isolate separate customer support conversations?

Module 7: ReAct Shipment Tracking Agent from Scratch

Company: OmniRoute Logistics (Supply Chain & Freight)

Role: Agentic Workflow Engineer Intern

Industry Context & Update: Freight dispatchers require autonomous agents capable of reasoning step-by-step and calling external live APIs (tracking, delay monitoring, ETA computation).

Problem Statement

Build a ReAct (Reason + Act) agent loop from scratch using LangGraph that alternates between reasoning steps, tool invocation (mock shipment API), and observation processing until answering user tracking queries.

Key Learning Outcomes

- Implement the fundamental ReAct cycle (Thought → Action → Observation).
- Connect custom Python functions as tools within an agent graph.

Tasks & Expected Deliverables

1. Define reasoning nodes, tool calling nodes, and conditional execution edges.
2. Test with multi-step freight tracking questions requiring external API queries.
3. Submit notebook (`react_agent.ipynb`) and execution trace log.

Knowledge Evaluation

1. Name the 3 phases of the ReAct execution loop.
2. Why is the observation phase critical before the agent takes its next reasoning step?

Module 8: Reflexion Agent Self-Critique Advisory Loop

Company: Apex Strategy Consulting (Management Advisory)

Role: AI Strategy Advisory Intern

Industry Context & Update: Top-tier management consultancies automate draft deliverable quality control using self-reflection patterns (Reflexion), iteratively critiquing and polishing strategy recommendations.

Problem Statement

Construct a two-node LangGraph loop (Responder & Reviser) with max-iteration control that drafts a market entry brief, evaluates its own output against advisory standards, and refines the draft iteratively.

Key Learning Outcomes

- Implement self-reflection and iterative critique patterns in LLM graphs.
- Apply bounded iteration control guards to prevent infinite loop execution.

Tasks & Expected Deliverables

1. Build a responder node and critique/reviser node loop.
2. Log each iteration's draft and self-critique score.
3. Submit notebook (`reflexion_consulting.ipynb`) showing at least 2 critique-revision cycles.

Knowledge Evaluation

1. What risks exist if a self-critique loop lacks a `MAX_ITERATIONS` limit?
2. How does automated self-critique differ from human-in-the-loop validation?

Module 9: Supervisor Multi-Agent System for Plant Operations

Company: Apex Dynamics Industrial (Smart Manufacturing)

Role: Multi-Agent Systems Intern

Industry Context & Update: Modern industrial manufacturing plants deploy hierarchical multi-agent teams where a Supervisor routes plant breakdown inquiries to specialist agents (Searcher, Log Coder, Validator).

Problem Statement

Construct a hierarchical multi-agent supervisor network in LangGraph where a central Supervisor orchestrates specialized worker agents (Research Agent, Python REPL Log Analyzer, Quality Validator Agent).

Key Learning Outcomes

- Build multi-agent orchestration graphs with dynamic routing.
- Enforce schema-validated task delegation across specialized sub-agents.

Tasks & Expected Deliverables

1. Implement Supervisor, Coder, Researcher, and Validator nodes.
2. Route a mixed query ("Research breakdown causes on Line 3 and analyze sensor logs") end-to-end.
3. Submit notebook (`supervisor_multiagent.ipynb`) and architecture diagram.

Knowledge Evaluation

1. Why choose a supervisor-worker architecture over a single large monolithic prompt?
2. What quality failures does the Validator node protect against?

Module 10: Human-in-the-Loop Trade Approval Workflow

Company: QuantCapital Trading (Financial Services)

Role: AI Risk & Governance Intern

Industry Context & Update: Autonomous financial trading execution requires mandatory human sign-off checkpoints (HITL) before submitting high-value stock trades or capital transfers.

Problem Statement

Build a LangGraph workflow incorporating `interrupt()` checkpoints that prepares stock order payloads, halts graph execution for human review, and resumes or halts execution based on manual approval.

Key Learning Outcomes

- Implement Human-in-the-Loop (HITL) interrupt and resume patterns in LangGraph.
- Ensure regulatory compliance in automated execution systems.

Tasks & Expected Deliverables

1. Build trade preparation graph with an explicit interrupt state node.
2. Execute two test scenarios: one approved by human input and one rejected.
3. Submit notebook (`hitl_trade_approval.ipynb`) and interactive run logs.

Knowledge Evaluation

1. Why is state machine interruption preferred over blocking `input()` calls in web applications?
2. How does LangGraph maintain state during interrupted execution sessions?

Module 11: Text Chunking Benchmark for Medical Device Manuals

Company: BioMedix Devices (Medical Equipment)

Role: RAG Evaluation & Benchmarking Intern

Industry Context & Update: Technical support for medical devices demands high precision when searching Instructions for Use (IFU) manuals. The choice of chunking strategy directly impacts retrieval recall and safety.

Problem Statement

Implement and systematically compare 4 text chunking strategies (Fixed-Size, Recursive Character, Sliding Window, and Semantic Chunking) on a complex medical device manual, evaluating downstream QA precision.

Key Learning Outcomes

- Implement advanced text splitting techniques in LangChain.
- Evaluate vector retrieval recall and precision across chunking strategies.

Tasks & Expected Deliverables

1. Process a sample device manual using 4 distinct chunking splitters.
2. Run 5 benchmark queries across all 4 vector indexes and compare accuracy.
3. Submit notebook (`chunking_benchmark.ipynb`) and comparative analysis report.

Knowledge Evaluation

1. Why does semantic chunking often outperform fixed-character splitting for technical manuals?
2. Trade-offs between small chunk sizes (high precision) and large chunk sizes (high context)?

Module 12: Custom MCP Server for Telecom Configuration Management

Company: TelcoNet Systems (Telecommunications)

Role: MCP Protocol Developer Intern

Industry Context & Update: Telecom Network Operations Centers (NOCs) leverage the open Model Context Protocol (MCP) standard to safely expose configuration file operations to AI models via stdio transport.

Problem Statement

Build a Model Context Protocol (MCP) server in Python using the official MCP SDK exposing tool endpoints (`read_file`, `write_file`, `find_replace`, `list_dir`) scoped to network router configs.

Key Learning Outcomes

- Understand Model Context Protocol architecture (Server, Client, Host).
- Build stdio-based tool servers using decorators and JSON-RPC over stdio.

Tasks & Expected Deliverables

1. Create an MCP server exposing file tools scoped to a configuration folder.
2. Connect an MCP client (or Claude Desktop) and test tool calls.
3. Submit server code (`mcp_file_server.py`) and client interaction log.

Knowledge Evaluation

1. What structural problems does the open MCP standard solve compared to custom tool calling APIs?
2. How does stdio transport work in local MCP server environments?

Module 13: Full-Stack Vehicle Diagnostic App with FastAPI + Gemini + MCP

Company: DriveTech Motors (Automotive Services)

Role: Full-Stack GenAI Intern

Industry Context & Update: Modern automotive diagnostic centers connect dealership REST backends with LLM reasoning engines and MCP servers to query vehicle telemetry data seamlessly.

Problem Statement

Build a full-stack REST service in FastAPI where a `/query` endpoint receives vehicle complaints, invokes a Gemini LLM client, connects to an MCP diagnostic server tool, and returns structured diagnosis data.

Key Learning Outcomes

- Integrate FastAPI, Gemini LLM SDK, and Model Context Protocol (MCP) clients.
- Manage async lifespans and CORS middleware in production backend APIs.

Tasks & Expected Deliverables

1. Implement FastAPI backend connecting MCP vehicle diagnostic tool server.
2. Expose `POST /query` and `GET /tools` endpoints.
3. Submit API backend code and Postman/curl call logs.

Knowledge Evaluation

1. Trace request flow: `Frontend` → `FastAPI` → `MCP Client` → `MCP Server` → `LLM`.
2. Why is asynchronous context management essential for MCP client-server lifecycles?

Module 14: Personalization Engine with Mem0 Hybrid Memory

Company: WanderLust Global (Tourism & Hospitality)

Role: Personalization AI Intern

Industry Context & Update: Next-generation travel concierge bots combine short-term conversational context with long-term vector memory (Mem0) to recall persistent user preferences (e.g. seating, dietary needs, hotel tier).

Problem Statement

Develop a hybrid memory travel chatbot using Mem0 and a vector database that automatically extracts, stores, and retrieves traveler preferences across distinct user sessions.

Key Learning Outcomes

- Implement long-term memory extraction and vector indexing using Mem0.
- Combine long-term retrieved memory with active short-term prompt context.

Tasks & Expected Deliverables

1. Configure Mem0 memory store with a vector store backend.
2. Demonstrate preference retention across two separated user chat sessions.
3. Submit notebook (`mem0_travel_concierge.ipynb`) and memory retrieval log.

Knowledge Evaluation

1. How does hybrid memory differ from passing sliding-window chat history?
2. Why is vector search preferred over traditional key-value stores for long-term agent memory?

Module 15: Quantitative Math Reasoning Agent with JSON Output

Company: AeroSpace Dynamics (Aerospace Flight Operations)

Role: Flight Operations AI Intern

Industry Context & Update: Flight operations teams require strict, auditable quantitative calculations (fuel burn rate, payload balance, climb gradient) using reasoning models (DeepSeek-R1 / Gemini 2.0 Thinking) with structured JSON output.

Problem Statement

Develop a flight operations calculation agent using reasoning-focused models to solve complex multi-step physics/math word problems, enforcing step-by-step JSON outputs for audit logs.

Key Learning Outcomes

- Utilize reasoning-distilled models for quantitative problem solving.
- Enforce structured JSON output containing explicit step-by-step reasoning arrays.

Tasks & Expected Deliverables

1. Design prompt requiring structured math reasoning in valid JSON.
2. Evaluate on 5 flight-ops calculation problems and save output JSON files.
3. Submit notebook (`aerospace_math_agent.ipynb`) and generated JSON logs.

Knowledge Evaluation

1. Why are reasoning-distilled models better suited for multi-step math problems than standard chat models?
2. How should applications handle cases where reasoning models output invalid JSON?

Module 16: Dynamic Prompt Templating & Dual-LLM Benchmarking

Company: FlavorCraft Foods (Food & Beverage R&D)

Role: Prompt Engineering Intern

Industry Context & Update: Consumer food R&D teams evaluate competing LLM models (e.g. Gemini 1.5 vs Groq Llama-3.3) side-by-side using dynamic prompt templates to compare recipe formulation outputs.

Problem Statement

Build a dual-LLM comparison dashboard that formats a single dynamic prompt template and dispatches concurrent queries to two distinct LLM providers, rendering side-by-side outputs.

Key Learning Outcomes

- Design reusable system and human message prompt templates.
- Benchmark output quality across distinct LLM model backends.

Tasks & Expected Deliverables

1. Construct a shared `ChatPromptTemplate` with dynamic slots.
2. Execute concurrent model calls to Gemini and Groq/Llama endpoints.
3. Submit notebook (`dual_llm_comparison.ipynb`) showing side-by-side transcripts.

Knowledge Evaluation

1. Why is decoupling prompt templates from LLM execution logic considered an engineering best practice?
2. What architectural factors cause variance between model responses given identical prompt inputs?

Module 17: Cloud-Native Chat Persistence with Firebase Firestore

Company: QuickBite QSR (Quick Service Restaurants)

Role: Cloud Conversational AI Intern

Industry Context & Update: Fast-food chains require cloud-persisted chat sessions across mobile apps and self-service kiosks using scalable NoSQL databases like Google Cloud Firebase Firestore.

Problem Statement

Build an ordering assistant chatbot that saves and retrieves conversation history to/from Firebase Firestore NoSQL collections, allowing users to resume orders across devices seamlessly.

Key Learning Outcomes

- Integrate cloud NoSQL databases (Firestore) for conversation persistence.
- Structure conversational state and history in document-oriented collections.

Tasks & Expected Deliverables

1. Configure Firebase admin SDK and Firestore collection schemas.
2. Test closing and reopening sessions, verifying chat history reload from Firestore.
3. Submit notebook (`firebase_qsr_chat.ipynb`) and Firestore database console screenshot.

Knowledge Evaluation

1. Why choose cloud NoSQL stores (Firestore) over embedded local databases (SQLite) for mobile applications?
2. Describe your proposed Firestore document schema for user chat sessions.

Module 18: TypeScript MCP Weather Server for Node.js

Company: SkyWay Travel (Tourism Tech)

Role: TypeScript & Node.js AI Developer Intern

Industry Context & Update: Modern Node.js web applications leverage the TypeScript Model Context Protocol SDK and Zod validation schemas to build robust MCP tool servers for client models.

Problem Statement

Build a Model Context Protocol server in TypeScript/Node.js using `@modelcontextprotocol/sdk` and Zod that exposes a `get_weather_by_city` tool over stdio transport.

Key Learning Outcomes

- Implement MCP servers in TypeScript/Node.js ecosystems.
- Enforce runtime input validation using Zod schemas for tool parameters.

Tasks & Expected Deliverables

1. Create TypeScript MCP project with tool handlers and Zod validation schemas.
2. Verify stdio tool invocation from an MCP client host.
3. Submit TypeScript server codebase and execution log.

Knowledge Evaluation

1. Why is runtime schema validation (Zod) essential for tool inputs in MCP servers?
2. How does stdio transport differ from HTTP SSE transport in MCP implementations?

Module 19: Competitor Price Monitoring Agent with Serper API

Company: PriceWatch Retail (Market Intelligence)

Role: Market Intelligence AI Intern

Industry Context & Update: E-commerce analytics teams automate price tracking by chaining search APIs (Serper API), HTML scraper tools (BeautifulSoup), and LLM summarization chains.

Problem Statement

Build an automated research agent that takes a pricing query (e.g., "latest discount trends on wireless earbuds"), searches the web via Serper API, scrapes top result pages, and generates a structured competitive report.

Key Learning Outcomes

- Combine Search APIs, web scraping, and LLM text processing into an agent.
- Handle unstructured web content extraction and source citation compilation.

Tasks & Expected Deliverables

1. Construct research pipeline querying Serper API and scraping top URLs.
2. Synthesize multi-source information into a formatted report with source links.
3. Submit notebook (`market_research_agent.ipynb`) and sample research output report.

Knowledge Evaluation

1. Why is a structured Search API preferred over directly scraping search engine result pages?
2. What legal and technical challenges arise when scraping arbitrary web pages?

Module 20: Nested Subgraph Multi-Agent Architecture for Enterprise Claims

Company: OmniInsurance Group (Enterprise Insurance)

Role: Enterprise Agent Architect Intern

Industry Context & Update: Enterprise platforms manage complex workflows by nesting specialized multi-node subgraphs (e.g. damage assessment sub-workflow) inside outer supervisor graphs.

Problem Statement

Architect a two-level LangGraph system where an outer supervisor graph routes high-level claims processing, invoking a compiled multi-node damage-assessment subgraph as a single modular node.

Key Learning Outcomes

- Compose complex modular agent architectures using nested LangGraph subgraphs.
- Manage hierarchical state passing and parent-child state synchronization.

Tasks & Expected Deliverables

1. Build inner damage-assessment subgraph (Research → Estimate → Summarize).
2. Wrap subgraph and mount as a node inside an outer supervisor graph.
3. Submit notebook (`nested_subgraph_claims.ipynb`) and complete state log.

Knowledge Evaluation

1. What architectural advantages does nesting subgraphs offer over expanding a single giant graph?
2. How is state isolated and communicated between outer graphs and inner subgraphs in LangGraph?